

**TESTE TÉCNICO – CORIS**  
**PROCESSO SELETIVO PARA ANALISTA DESENVOLVEDOR PLENO**

**CARLOS GUILHERME FONTES PEREIRA**

**SEGURO VIAGEM:**  
**SISTEMA DE GESTÃO DE APÓLICES COM**  
**DASHBOARD DE INDICADORES**

Relatório técnico do projeto

**CARLOS GUILHERME FONTES PEREIRA**

**SEGURO VIAGEM: SISTEMA DE GESTÃO DE APÓLICES COM  
DASHBOARD DE INDICADORES**

Relatório técnico apresentado à CORIS como entrega do desafio técnico do processo seletivo para a vaga de Analista Desenvolvedor Pleno, descrevendo a solução desenvolvida, suas decisões de projeto e os resultados obtidos.

## RESUMO

Este relatório descreve o desenvolvimento de uma aplicação web para a gestão de apólices de seguro viagem, construída como resposta a um desafio técnico. A aplicação permite cadastrar, consultar, editar e excluir apólices, calculando o valor do seguro (prêmio) a partir do plano, do destino, da duração da viagem e da idade do segurado. Além do cadastro, foi construída uma *dashboard* com indicadores voltados a três públicos: a diretoria, o time de marketing, que avalia a conversão e o retorno das campanhas, e o time comercial, que acompanha as vendas. A solução utiliza Laravel no *backend*, React no *frontend* e MySQL como banco de dados relacional, com desenho de implantação na nuvem Microsoft Azure. O código foi organizado em camadas, seguindo os princípios SOLID e as práticas de Clean Code, e conta com testes automatizados. Como resultado, obteve-se um sistema funcional, executável com um único comando por meio de Docker, cujos indicadores respondem a perguntas reais de negócio.

**Palavras-chave:** seguro viagem; aplicação web; Laravel; React; dashboard; SOLID.

## LISTA DE FIGURAS

|    |                                                                             |    |
|----|-----------------------------------------------------------------------------|----|
| 1  | Tela de listagem de apólices . . . . .                                      | 8  |
| 2  | Tela de cadastro de apólice, com o prêmio calculado em tempo real . . . . . | 9  |
| 3  | Diagrama de entidade e relacionamento . . . . .                             | 10 |
| 4  | Visão geral da dashboard . . . . .                                          | 14 |
| 5  | Visão de marketing . . . . .                                                | 15 |
| 6  | Detalhe de uma campanha, semana a semana . . . . .                          | 16 |
| 7  | Visão comercial . . . . .                                                   | 17 |
| 8  | Visão de sinistros e atendimento . . . . .                                  | 18 |
| 9  | Desenho da solução na Microsoft Azure . . . . .                             | 20 |
| 10 | Publicação atual na Microsoft Azure . . . . .                               | 21 |

## LISTA DE TABELAS

|   |                                                    |    |
|---|----------------------------------------------------|----|
| 1 | Tecnologias utilizadas . . . . .                   | 7  |
| 2 | Tabelas do banco de dados . . . . .                | 11 |
| 3 | Fatores do cálculo do prêmio . . . . .             | 12 |
| 4 | Onde os princípios SOLID foram aplicados . . . . . | 13 |
| 5 | Fórmulas dos indicadores . . . . .                 | 18 |

## SUMÁRIO

|           |                                                    |           |
|-----------|----------------------------------------------------|-----------|
| <b>1</b>  | <b>Introdução</b>                                  | <b>5</b>  |
| 1.1       | Objetivos . . . . .                                | 5         |
| 1.2       | Organização do relatório . . . . .                 | 5         |
| <b>2</b>  | <b>Requisitos</b>                                  | <b>6</b>  |
| 2.1       | Requisitos funcionais . . . . .                    | 6         |
| 2.2       | Requisitos não funcionais . . . . .                | 6         |
| <b>3</b>  | <b>Tecnologias utilizadas</b>                      | <b>6</b>  |
| <b>4</b>  | <b>Arquitetura da solução</b>                      | <b>7</b>  |
| 4.1       | Visão geral . . . . .                              | 7         |
| 4.2       | Organização do backend . . . . .                   | 7         |
| 4.3       | Organização do frontend . . . . .                  | 8         |
| <b>5</b>  | <b>Modelo de dados</b>                             | <b>9</b>  |
| <b>6</b>  | <b>Regras de negócio</b>                           | <b>11</b> |
| 6.1       | Cálculo do prêmio . . . . .                        | 11        |
| 6.2       | Validações . . . . .                               | 12        |
| <b>7</b>  | <b>Aplicação dos princípios SOLID e Clean Code</b> | <b>12</b> |
| <b>8</b>  | <b>Dashboard de indicadores</b>                    | <b>14</b> |
| 8.1       | Visão geral . . . . .                              | 14        |
| 8.2       | Marketing . . . . .                                | 14        |
| 8.3       | Comercial . . . . .                                | 16        |
| 8.4       | Sinistros e atendimento . . . . .                  | 17        |
| 8.5       | Fórmulas dos indicadores . . . . .                 | 18        |
| 8.6       | Dados de demonstração . . . . .                    | 19        |
| <b>9</b>  | <b>Testes</b>                                      | <b>19</b> |
| <b>10</b> | <b>Execução e implantação</b>                      | <b>19</b> |
| 10.1      | Execução local . . . . .                           | 19        |
| 10.2      | Implantação na Microsoft Azure . . . . .           | 19        |
| <b>11</b> | <b>Considerações finais</b>                        | <b>21</b> |
|           | <b>REFERÊNCIAS</b>                                 | <b>21</b> |

## 1 INTRODUÇÃO

O seguro viagem é contratado por quem vai viajar para cobrir despesas médicas, extravio de bagagem, cancelamento da viagem e outros imprevistos. Para a seguradora, cada contrato é uma *apólice*: um documento que registra quem é o segurado, para onde vai, por quanto tempo, qual plano contratou e quanto pagou por isso. O valor pago é chamado de *prêmio*.

O desafio proposto consistiu em desenvolver uma aplicação capaz de realizar o ciclo completo de cadastro dessas apólices (leitura, cadastro, edição e exclusão), aplicando boas práticas de desenvolvimento e prevendo a implantação na nuvem Microsoft Azure. Como diferencial, buscou-se ir além do cadastro e transformar os dados das apólices em informação útil para a tomada de decisão dos times de marketing e comercial, por meio de uma *dashboard* de indicadores.

### 1.1 Objetivos

O objetivo geral foi construir uma aplicação web para a gestão de apólices de seguro viagem, com cálculo automático do prêmio e indicadores de negócio. Os objetivos específicos foram:

- oferecer o cadastro completo de apólices, com validação dos dados e cálculo do prêmio em tempo real;
- modelar um banco de dados relacional que represente segurados, apólices e as informações de vendas, marketing, sinistros e atendimento;
- organizar o código em camadas, aplicando os princípios SOLID e as práticas de Clean Code;
- criar uma *dashboard* com visões específicas para cada time;
- garantir o funcionamento por meio de testes automatizados;
- projetar a implantação da solução com componentes da Microsoft Azure.

### 1.2 Organização do relatório

A Seção 2 apresenta os requisitos e a Seção 3, as tecnologias escolhidas. A arquitetura é descrita na Seção 4 e o modelo de dados, na Seção 5. As regras de negócio estão na Seção 6 e a aplicação dos princípios SOLID e Clean Code, na Seção 7. A Seção 8 descreve a *dashboard*, a Seção 9 trata dos testes e a Seção 10, da execução e da implantação. Por fim, a Seção 11 traz as considerações finais.

## 2 REQUISITOS

Os requisitos foram definidos a partir do enunciado do desafio e divididos entre funcionais, que descrevem o que o sistema faz, e não funcionais, que descrevem como ele deve ser construído.

### 2.1 Requisitos funcionais

- **RF01** – Listar as apólices, com busca por nome, CPF, e-mail ou número da apólice, filtro por situação e paginação.
- **RF02** – Cadastrar uma apólice, informando os dados do segurado, o destino, o plano e o período da viagem.
- **RF03** – Calcular o prêmio automaticamente, exibindo o valor enquanto o formulário é preenchido.
- **RF04** – Editar uma apólice, recalculando o prêmio, e cancelá-la ou reativá-la.
- **RF05** – Excluir uma apólice, mantendo o registro para fins de auditoria.
- **RF06** – Exibir indicadores de vendas, marketing, sinistros e atendimento, filtrados por período.

### 2.2 Requisitos não funcionais

- **RNF01** – *Frontend* em React, versão 17 ou superior.
- **RNF02** – *Backend* em PHP, conforme autorização da equipe de tecnologia da empresa para este candidato.
- **RNF03** – Banco de dados relacional.
- **RNF04** – Código seguindo os princípios SOLID e as práticas de Clean Code.
- **RNF05** – Desenho da solução com componentes da Microsoft Azure.
- **RNF06** – Interface em português, com valores em reais e datas no formato dia/mês/ano.

## 3 TECNOLOGIAS UTILIZADAS

A Tabela 1 resume as tecnologias adotadas. A escolha priorizou ferramentas maduras, com boa documentação e amplamente usadas no mercado.

Tabela 1 – Tecnologias utilizadas

| Camada          | Tecnologia e motivo da escolha                                                                                       |
|-----------------|----------------------------------------------------------------------------------------------------------------------|
| <i>Backend</i>  | PHP 8.3 com Laravel 12: oferece rotas, validação, acesso ao banco (Eloquent), migrações e testes de forma integrada. |
| <i>Frontend</i> | React 18 com Vite e React Router; Recharts para os gráficos.                                                         |
| Banco de dados  | MySQL 8 em produção; SQLite para rodar localmente sem instalar um servidor.                                          |
| Testes          | PHPUnit, já integrado ao Laravel.                                                                                    |
| Ambiente        | Docker Compose, que sobe o banco, a API e o <i>frontend</i> com um comando.                                          |
| Nuvem           | Microsoft Azure: Static Web Apps, App Service, Database for MySQL e Key Vault.                                       |

Fonte: elaborado pelo autor (2026).

## 4 ARQUITETURA DA SOLUÇÃO

### 4.1 Visão geral

A aplicação é dividida em duas partes independentes que se comunicam por HTTP, trocando dados no formato JSON. O *backend* é uma API REST responsável pelas regras de negócio e pelo acesso ao banco de dados. O *frontend* é uma aplicação React que apenas exibe as informações e envia os formulários, sem conter regras de negócio. Essa separação permite que cada parte evolua e seja publicada de forma independente.

### 4.2 Organização do backend

O *backend* foi organizado em camadas, e cada camada tem uma única responsabilidade. Quando uma requisição chega, ela percorre o seguinte caminho:

1. a **rota** (`routes/api.php`) identifica qual ação deve ser executada;
2. o **Form Request** (`app/Http/Requests`) valida os dados recebidos; se houver erro, a resposta é devolvida imediatamente com a mensagem de cada campo;
3. o **Controller** (`app/Http/Controllers/Api`) recebe os dados validados e chama o serviço correspondente;
4. o **Service** (`app/Services`) aplica as regras de negócio, usando a **Calculadora** de prêmio e os **Models**, que representam as tabelas;
5. o **Resource** (`app/Http/Resources`) converte o resultado para o formato JSON esperado pelo *frontend*.



Por exemplo, ao emitir uma apólice, o formulário envia os dados para a rota POST /api/apolices. O `SalvarApoliceRequest` valida o CPF, as datas e os demais campos; o `ApoliceController` repassa os dados ao `ApoliceService`, que, dentro de uma transação, localiza ou cria o segurado pelo CPF, calcula o prêmio, gera o número da apólice e grava o registro. A resposta volta com o código HTTP 201 (criado).

### 4.3 Organização do frontend

No *frontend*, cada tela fica em `src/pages`, os componentes reutilizáveis em `src/components` e toda a comunicação com a API em `src/api`. Nenhuma tela chama a API diretamente: todas usam as funções desse módulo, que padronizam o envio dos dados e o tratamento de erros. A formatação de moeda, datas e CPF fica concentrada em `src/utils/format.js`.

As Figuras 1 e 2 mostram as telas de listagem e de cadastro. No cadastro, o prêmio aparece à direita e é recalculado enquanto os campos são preenchidos.

Figura 1 – Tela de listagem de apólices

| Nº APÓLICE        | SEGURADO                                     | DESTINO / PLANO             | VIGÊNCIA                           | PRÊMIO       | STATUS |
|-------------------|----------------------------------------------|-----------------------------|------------------------------------|--------------|--------|
| CRS-2026-BE1A1E29 | <b>Mariana Alves Costa</b><br>529.982.247-25 | Nacional<br>Essencial       | 23/12/2026 a 26/12/2026<br>4 dias  | R\$ 25,80    | Ativa  |
| CRS-2026-E2344813 | <b>Fernanda Rocha</b><br>714.602.380-01      | Nacional<br>Essencial       | 26/09/2026 a 30/09/2026<br>5 dias  | R\$ 32,25    | Ativa  |
| CRS-2026-0C27D48A | <b>Ricardo Menezes</b><br>168.995.350-09     | Ásia<br>Premium             | 23/11/2026 a 13/12/2026<br>21 dias | R\$ 1.256,85 | Ativa  |
| CRS-2026-8A7C455F | <b>Ana Beatriz Lima</b><br>590.533.447-05    | América do Sul<br>Essencial | 29/09/2026 a 05/10/2026<br>7 dias  | R\$ 90,30    | Ativa  |
| CRS-2026-7AACB021 | <b>João Pedro Santos</b><br>111.444.777-35   | América do Norte<br>Premium | 24/10/2026 a 07/11/2026<br>15 dias | R\$ 1.340,64 | Ativa  |
| CRS-2026-7DD5309D | <b>Mariana Alves Costa</b><br>529.982.247-25 | Europa<br>Plus              | 04/10/2026 a 18/10/2026<br>15 dias | R\$ 485,55   | Ativa  |
| CRS-2026-6D9EC45C | <b>Renata Silva</b><br>449.197.755-04        | Europa<br>Premium           | 27/09/2026 a 02/12/2026<br>67 dias | R\$ 3.475,29 | Ativa  |
| CRS-2026-0BE8B5ED | <b>Patrícia Almeida</b><br>690.014.779-84    | América do Norte<br>Plus    | 24/09/2026 a 03/10/2026<br>10 dias | R\$ 348,60   | Ativa  |

Fonte: elaborado pelo autor (2026).

Figura 2 – Tela de cadastro de apólice, com o prêmio calculado em tempo real

**Coris Seguros**  
Gestão de Apólices

Apólices Dashboard

### Nova apólice

O valor do prêmio é calculado pelo plano, destino, dias de viagem e idade do segurado.

**Dados do segurado**

Nome completo  
Carlos Guilherme

CPF  
123.456.789-09

Data de nascimento  
05/10/1995

E-mail  
carlos@email.com

**Viagem e cobertura**

Destino  
Europa

Plano  
Plus — R\$ 24,90/dia

Início da vigência  
10/04/2026

Fim da vigência  
10/13/2026

**Prêmio estimado**  
**R\$ 323,70**  
10 dias de cobertura

**Plano Plus**  
Cobertura médica de até R\$ 60.000,00

Emitir apólice

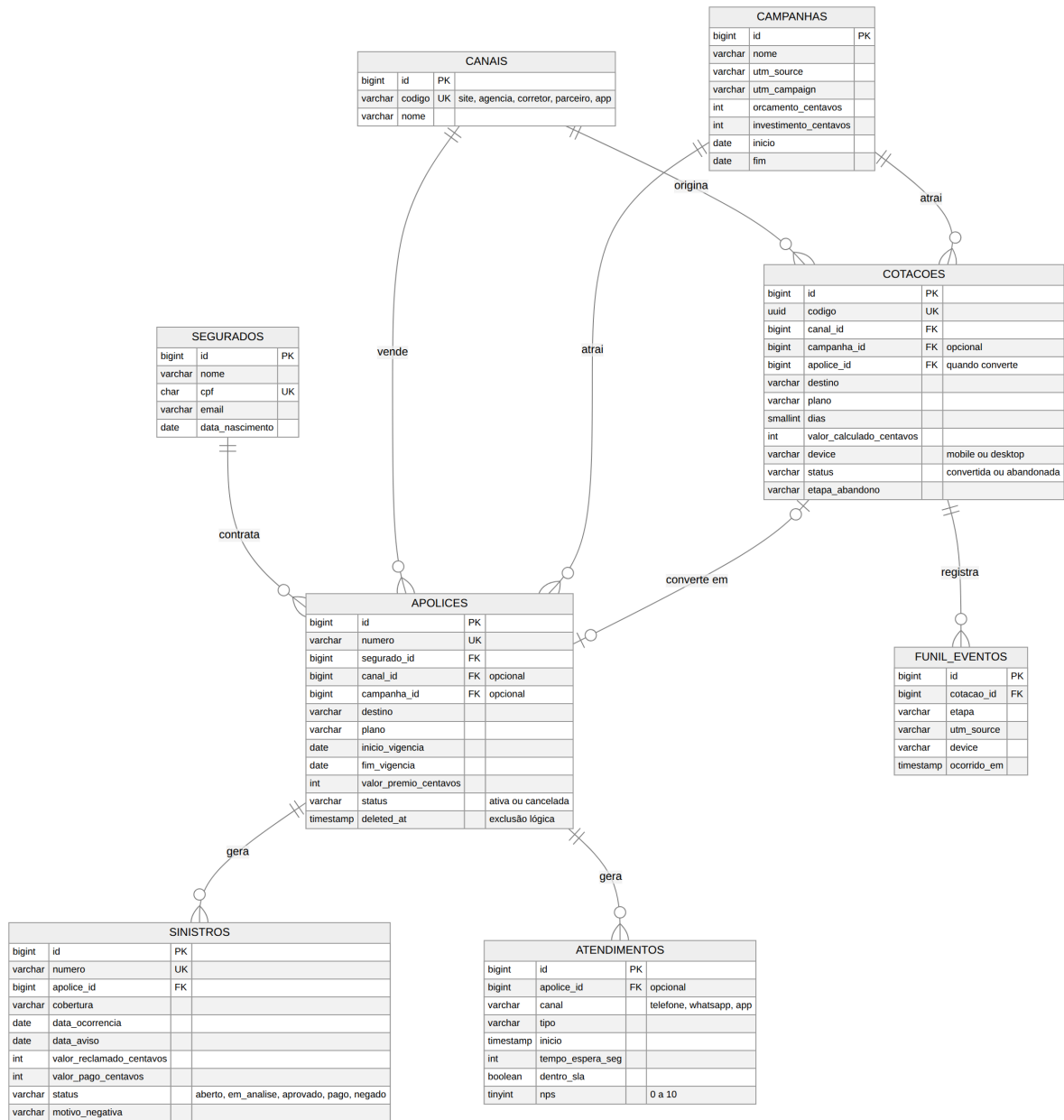
Voltar

Fonte: elaborado pelo autor (2026).

## 5 MODELO DE DADOS

O banco de dados possui oito tabelas. As tabelas segurados e apólices formam o cadastro principal. As outras seis armazenam as informações usadas pela *dashboard*: os canais de venda, as campanhas de marketing, as cotações feitas pelos clientes e as etapas que eles percorreram, os sinistros e os atendimentos da central. A Figura 3 apresenta o diagrama de entidade e relacionamento.

Figura 3 – Diagrama de entidade e relacionamento



Fonte: elaborado pelo autor (2026).

A Tabela 2 descreve o conteúdo de cada tabela.

Tabela 2 – Tabelas do banco de dados

| Tabela        | O que armazena                                                                       |
|---------------|--------------------------------------------------------------------------------------|
| segurados     | Pessoas que contratam o seguro. O CPF é único.                                       |
| apolices      | Os contratos: destino, plano, vigência, prêmio, situação, canal e campanha da venda. |
| canais        | Por onde a apólice foi vendida: site, agências, corretores, parceiros e aplicativo.  |
| campanhas     | Campanhas de marketing, com origem, orçamento e valor investido.                     |
| cotacoes      | Cada simulação de preço, convertida em apólice ou abandonada.                        |
| funil_eventos | Cada etapa que o cliente percorreu durante a cotação.                                |
| sinistros     | Ocorrências cobertas pela apólice, com valor reclamado, valor pago e situação.       |
| atendimentos  | Contatos com a central 24 horas, com tempo de espera e nota de satisfação.           |

Fonte: elaborado pelo autor (2026).

Três decisões de modelagem merecem destaque. A primeira é que todos os valores em dinheiro são armazenados como números inteiros de centavos: o prêmio de R\$ 323,70 é gravado como 32370. Os números de ponto flutuante não representam valores decimais com exatidão, o que poderia gerar diferenças de centavos, inaceitáveis em valores financeiros. A segunda é que o segurado fica em uma tabela separada da apólice, identificado pelo CPF, de modo que a mesma pessoa pode ter várias apólices sem que seus dados sejam repetidos. A terceira é a *exclusão lógica*: ao excluir uma apólice, o registro não é apagado, apenas recebe a data de exclusão, deixando de aparecer no sistema, mas permanecendo disponível para auditoria, como exige a atividade de seguros.

As tabelas são criadas por *migrations*, arquivos que descrevem a estrutura do banco e permitem recriá-lo em qualquer ambiente. As tabelas da *dashboard* foram adicionadas em *migrations* novas, sem alterar as existentes.

## 6 REGRAS DE NEGÓCIO

### 6.1 Cálculo do prêmio

O prêmio é calculado pela multiplicação de quatro fatores:

$$\text{prêmio} = \text{diária do plano} \times \text{dias de viagem} \times \text{fator do destino} \times \text{fator da idade}$$

Os dias contam o primeiro e o último dia da viagem, e a idade é calculada na data de início da viagem. Os valores de cada fator estão na Tabela 3.

Tabela 3 – Fatores do cálculo do prêmio

|           |           | Destino               | Fator |                 |       |
|-----------|-----------|-----------------------|-------|-----------------|-------|
| Plano     | Diária    |                       |       | Idade           | Fator |
| Essencial | R\$ 12,90 | Nacional              | 50%   | até 59 anos     | 100%  |
| Plus      | R\$ 24,90 | América do Sul        | 100%  | 60 a 74 anos    | 160%  |
| Premium   | R\$ 39,90 | Europa                | 130%  | 75 anos ou mais | 250%  |
|           |           | América do Norte      | 140%  |                 |       |
|           |           | Ásia, África, Oceania | 150%  |                 |       |

Fonte: elaborado pelo autor (2026).

Como exemplo, uma viagem de 10 dias para a Europa no plano Plus, feita por um segurado de 31 anos, resulta em  $2.490 \times 10 \times 1,30 \times 1,00 = 32.370$  centavos, ou seja, R\$ 323,70. O arredondamento é feito sempre com números inteiros, para o centavo mais próximo.

## 6.2 Validações

Antes de gravar uma apólice, o sistema verifica que todos os campos foram preenchidos, que o CPF possui dígitos verificadores válidos, que o fim da viagem não é anterior ao início e que a vigência não passa de 365 dias. No cadastro, o início da viagem não pode ser anterior à data atual; na edição, essa regra só é aplicada se a data de início for alterada, para que uma apólice já em vigor continue editável. Uma apólice cancelada só pode ser alterada para voltar a ficar ativa. Quando alguma regra não é atendida, a API responde com o código HTTP 422 e uma mensagem em português para cada campo, exibida abaixo do campo correspondente no formulário.

## 7 APLICAÇÃO DOS PRINCÍPIOS SOLID E CLEAN CODE

SOLID é um conjunto de cinco princípios de projeto orientado a objetos que tornam o código mais fácil de entender, testar e modificar (MARTIN, 2019). Clean Code reúne práticas para que o código seja legível por outras pessoas (MARTIN, 2009). A Tabela 4 aponta onde cada princípio SOLID foi aplicado.

Tabela 4 – Onde os princípios SOLID foram aplicados

| Princípio                    | Aplicação no projeto                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidade única (S)   | Cada etapa do cadastro é uma classe diferente: <code>SalvarApoliceRequest</code> valida, <code>ApoliceController</code> trata a requisição, <code>ApoliceService</code> aplica a regra de negócio, <code>CalculadoraPremioViagem</code> calcula e <code>ApoliceResource</code> formata a resposta. Na <i>dashboard</i> , o mesmo ocorre com <code>DashboardController</code> , <code>DashboardService</code> , <code>Metricas</code> e <code>Periodo</code> . |
| Aberto/fechado (O)           | Uma nova regra de preço é criada como uma nova classe que implementa a interface <code>CalculadoraPremio</code> , sem alterar o serviço. Um novo plano ou destino é uma nova opção no enum correspondente, e o <i>frontend</i> o exibe sem alteração.                                                                                                                                                                                                         |
| Substituição de Liskov (L)   | Qualquer classe que implemente <code>CalculadoraPremio</code> pode substituir a atual, pois o serviço conhece apenas o contrato da interface.                                                                                                                                                                                                                                                                                                                 |
| Segregação de interfaces (I) | A interface <code>CalculadoraPremio</code> possui um único método, <code>calcular</code> .                                                                                                                                                                                                                                                                                                                                                                    |
| Inversão de dependência (D)  | O <code>ApoliceService</code> recebe a interface no construtor, e não a classe concreta; a escolha da implementação é feita no <code>AppServiceProvider</code> .                                                                                                                                                                                                                                                                                              |

Fonte: elaborado pelo autor (2026).

O trecho abaixo mostra a inversão de dependência. O serviço depende apenas da interface, e o *service provider* do Laravel informa qual implementação deve ser usada. Para trocar a regra de preço, basta alterar essa única linha.

```
// app/Services/ApoliceService.php
public function __construct(private readonly CalculadoraPremio $calculadora) {}

// app/Providers/AppServiceProvider.php
$this->app->bind(CalculadoraPremio::class, CalculadoraPremioViagem::class);
```

Quanto ao Clean Code, as classes e os métodos usam o vocabulário do negócio, como `ApoliceService::criar` e `Metricas::sinistralidade`. Os métodos são curtos e têm uma tarefa cada: a criação da apólice, por exemplo, delega a outros métodos o registro do segurado, o cálculo do prêmio e a geração do número. Não há números soltos no código: preços e fatores ficam nos enums `Plano` e `Destino`, e limites em constantes como `VIGENCIA_MAXIMA_DIAS`. As fórmulas da *dashboard* estão todas em uma única classe, `Metricas`, de modo que todas as telas usam a mesma conta. Os erros da API são tratados em um único lugar, `bootstrap/app.php`, e os nomes dos testes descrevem a regra verificada, como `test_apolice_cancelada_so_pode_ser_reativada`.

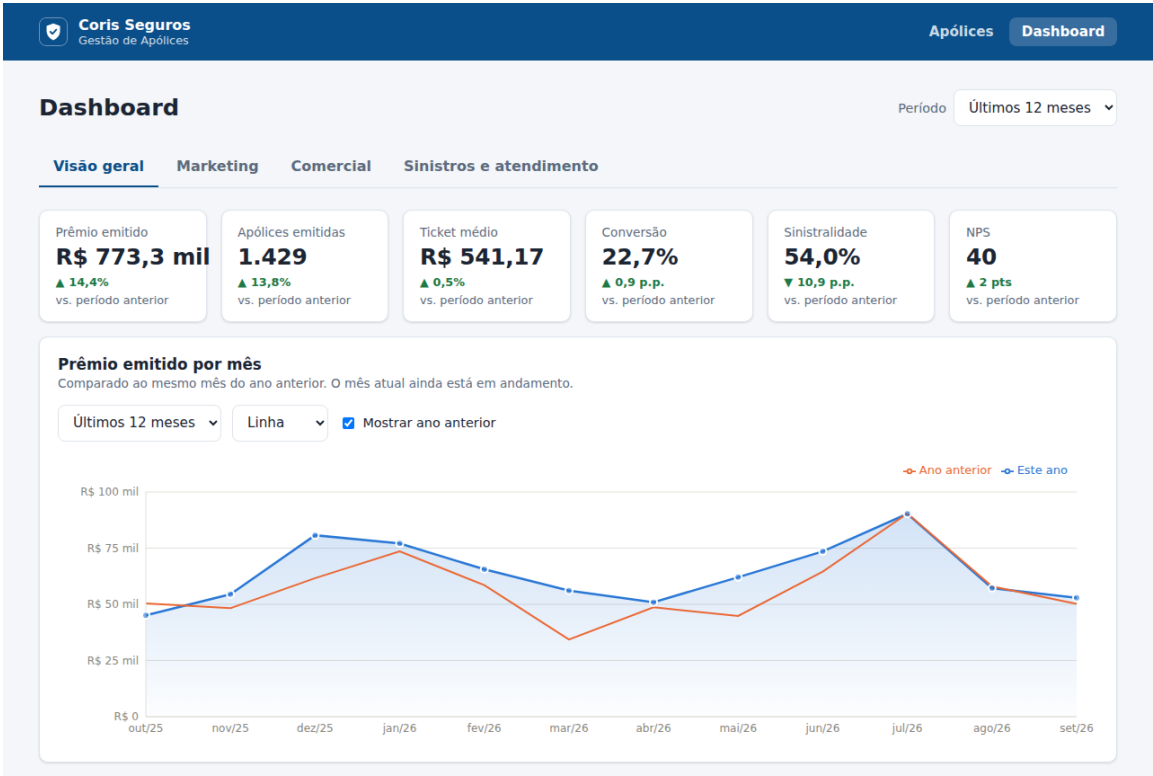
8 DASHBOARD DE INDICADORES

A *dashboard* foi o principal diferencial do projeto. Enquanto o cadastro atende à operação do dia a dia, a *dashboard* transforma os dados em informação para a tomada de decisão. Ela foi dividida em quatro visões, cada uma pensada para as perguntas de um time, e todas podem ser filtradas pelos últimos 30 ou 90 dias ou pelos últimos 12 ou 24 meses.

8.1 Visão geral

A visão geral é voltada à diretoria. Ela mostra seis indicadores (prêmio emitido, apólices, ticket médio, conversão, sinistralidade e NPS), cada um comparado com o período anterior de mesmo tamanho, e um gráfico do prêmio mês a mês comparado com o mesmo mês do ano anterior (Figura 4). O gráfico pode exibir 6 ou 12 meses, em linha ou em colunas.

Figura 4 – Visão geral da dashboard



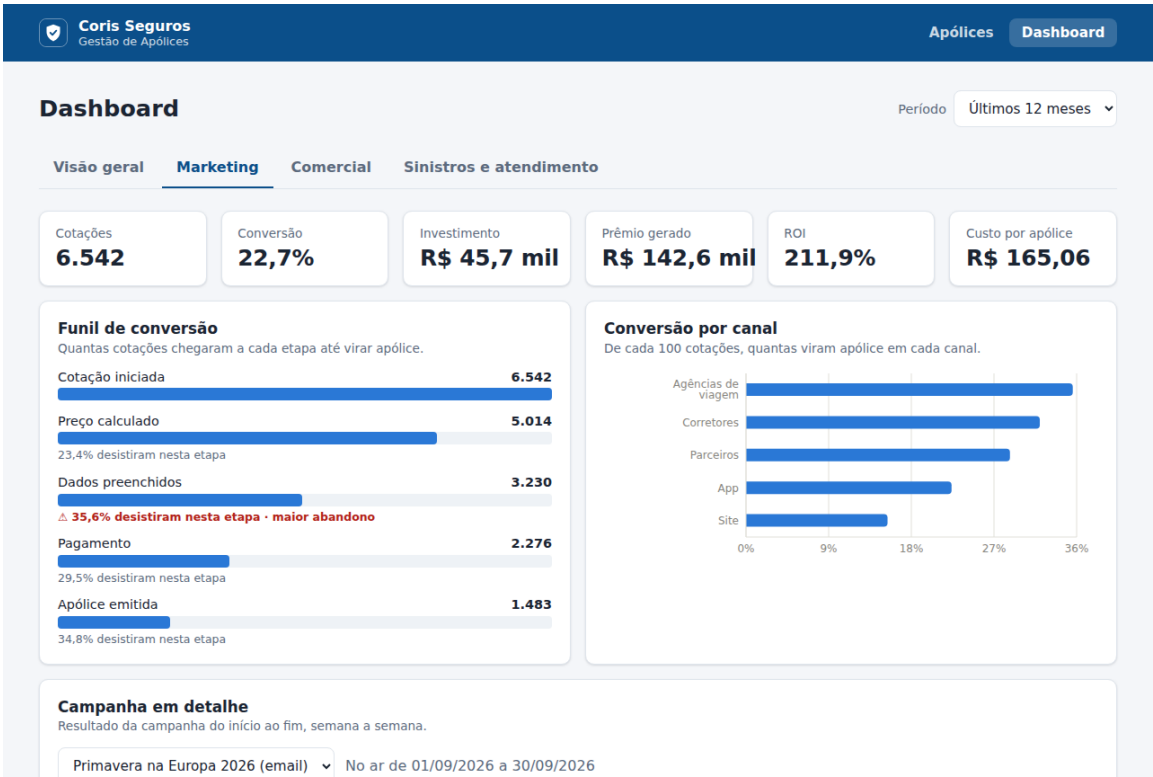
Fonte: elaborado pelo autor (2026).

8.2 Marketing

O time de marketing precisa saber se os clientes que chegam estão comprando e se o dinheiro investido em campanhas está voltando. Por isso, essa visão mostra a quantidade de cotações, a taxa de conversão, o valor investido, o prêmio gerado, o retorno sobre o investimento (ROI) e o custo para vender cada apólice (Figura 5). O funil de conversão mostra em que etapa da cotação os clientes desistem, destacando a de maior abandono, e a conversão por canal revela,

por exemplo, que as agências convertem proporcionalmente mais do que o site.

Figura 5 – Visão de marketing



Fonte: elaborado pelo autor (2026).

Cada campanha pode ser analisada em detalhe (Figura 6): ao escolhê-la, são exibidos o investimento, o prêmio gerado, o ROI, o custo por apólice e a conversão, além de um gráfico com as cotações e as apólices de cada semana em que ela esteve no ar. Uma tabela compara todas as campanhas do período, e um gráfico mostra com quantos dias de antecedência os clientes compram o seguro, informação útil para decidir quando lançar uma campanha.



Figura 6 – Detalhe de uma campanha, semana a semana

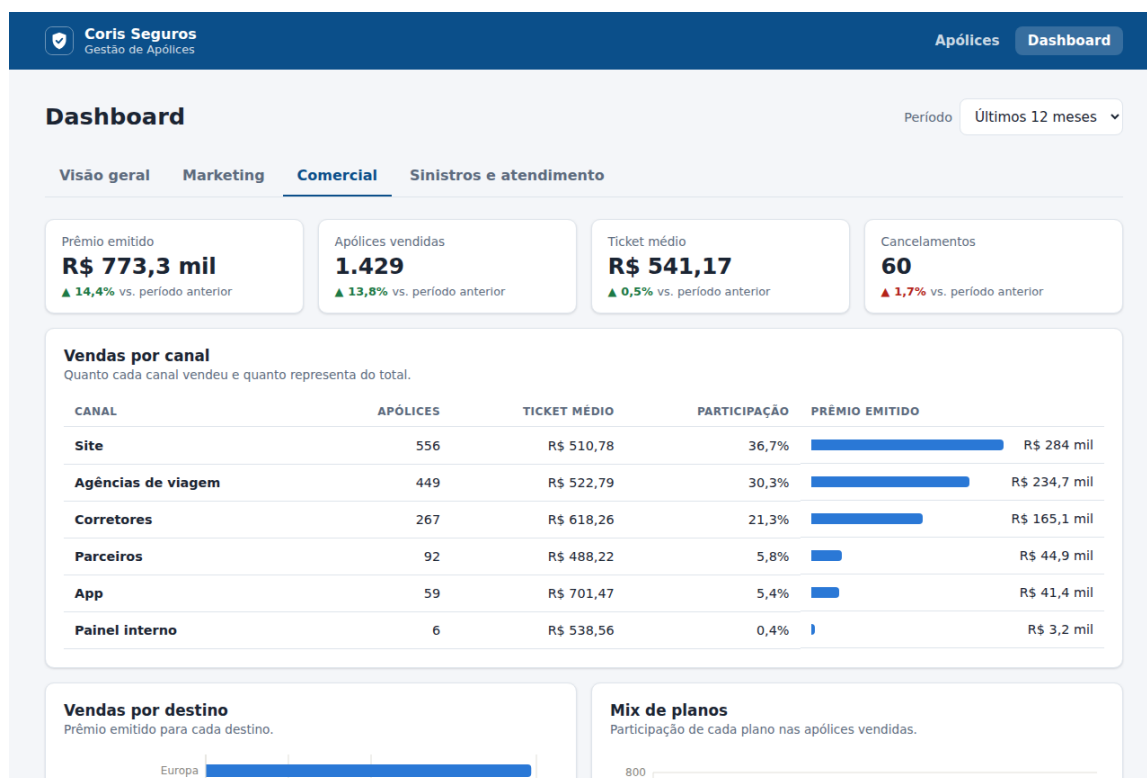


Fonte: elaborado pelo autor (2026).

8.3 Comercial

O time comercial acompanha as vendas em valores. Essa visão mostra o prêmio emitido, as apólices vendidas, o ticket médio e os cancelamentos, comparados ao período anterior; a participação de cada canal no total vendido; as vendas por destino; e a participação e o ticket médio de cada plano (Figura 7).

Figura 7 – Visão comercial

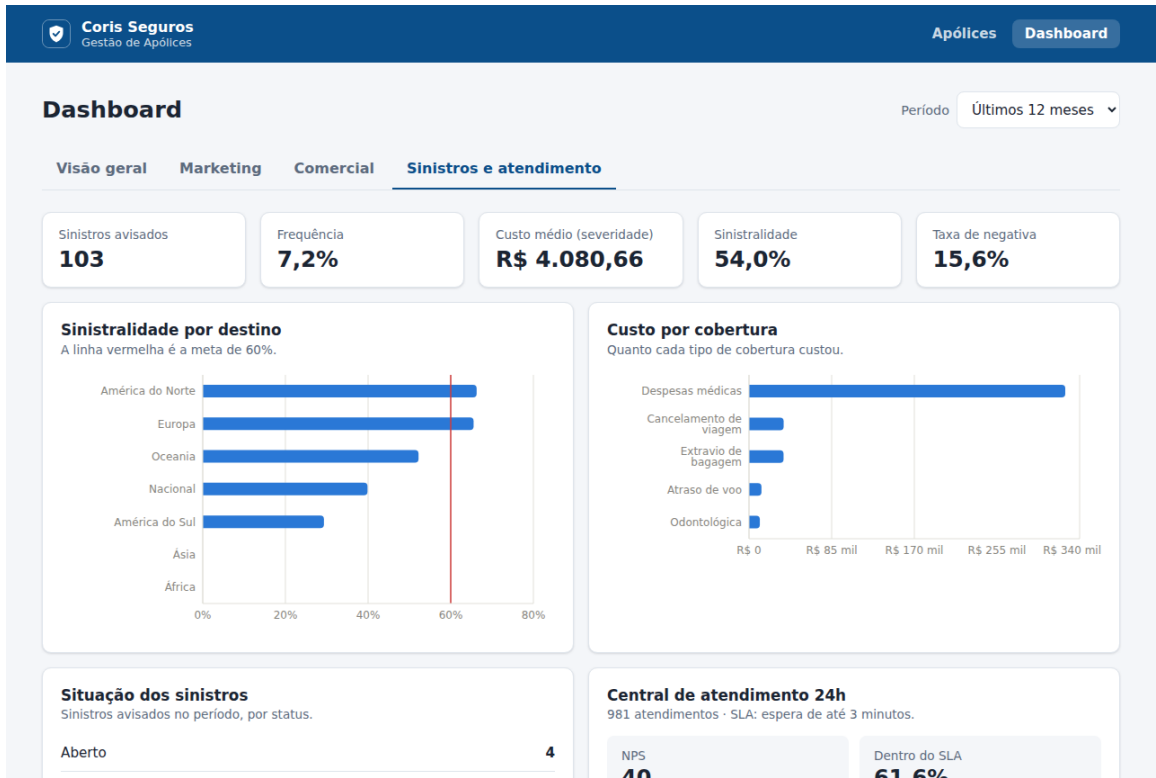


Fonte: elaborado pelo autor (2026).

## 8.4 Sinistros e atendimento

A última visão acompanha o custo dos sinistros e a qualidade do atendimento: frequência e custo médio dos sinistros, sinistralidade por destino comparada a uma meta de 60%, custo por tipo de cobertura, situação dos sinistros e, na central de atendimento, o NPS e o percentual de atendimentos dentro do prazo (Figura 8).

Figura 8 – Visão de sinistros e atendimento



Fonte: elaborado pelo autor (2026).

8.5 Fórmulas dos indicadores

Todas as fórmulas estão centralizadas na classe *Metricas*, apresentadas na Tabela 5.

Tabela 5 – Fórmulas dos indicadores

| Indicador         | Fórmula                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------|
| Ticket médio      | $\text{prêmio emitido} \div \text{número de apólices}$                                      |
| Conversão         | $\text{cotações convertidas} \div \text{total de cotações}$                                 |
| ROI da campanha   | $(\text{prêmio gerado} - \text{investimento}) \div \text{investimento}$                     |
| Custo por apólice | $\text{investimento} \div \text{apólices vendidas pela campanha}$                           |
| Prêmio ganho      | $\text{prêmio} \times (\text{dias de viagem dentro do período} \div \text{dias da viagem})$ |
| Sinistralidade    | $\text{custo dos sinistros} \div \text{prêmio ganho}$                                       |
| Frequência        | $\text{sinistros avisados} \div \text{apólices emitidas}$                                   |
| Taxa de negativa  | $\text{sinistros negados} \div \text{sinistros finalizados}$                                |
| NPS               | $\% \text{ de notas 9 e 10} - \% \text{ de notas de 0 a 6}$                                 |

Fonte: elaborado pelo autor (2026).

A sinistralidade usa o prêmio *ganho*, e não o prêmio emitido. Uma apólice vendida hoje para uma viagem daqui a dois meses ainda não correu nenhum risco; se todo o seu prêmio fosse considerado de imediato, a sinistralidade de períodos curtos ficaria distorcida.

## 8.6 Dados de demonstração

Para que a *dashboard* pudesse ser avaliada, foi criado um gerador de dados de demonstração que produz 24 meses de histórico: cerca de 2.800 apólices, 12.800 cotações, 180 sinistros e 1.800 atendimentos. Os dados seguem padrões realistas, como vendas maiores em julho, dezembro e janeiro, crescimento de 12% ao ano, campanhas sazonais e conversão menor no celular. O gerador usa uma semente fixa, de modo que produz sempre os mesmos dados, e pode ser executado mais de uma vez sem duplicar informações.

## 9 TESTES

O projeto possui 40 testes automatizados, escritos com PHPUnit e executados com o comando `php artisan test`. Eles se dividem em dois grupos:

- **testes unitários**, que verificam partes isoladas do código: o cálculo do prêmio em diferentes combinações de plano, destino e idade; a validação do CPF; e as fórmulas dos indicadores;
- **testes de API**, que fazem requisições reais à aplicação e verificam o cadastro, a busca, a paginação, a edição com recálculo do prêmio, a regra da apólice cancelada, a exclusão lógica, as mensagens de validação e os números de cada visão da *dashboard*.

Nos testes de API, a data atual é fixada e o banco de dados é recriado a cada teste, para que os resultados não dependam do dia em que são executados nem da ordem de execução.

## 10 EXECUÇÃO E IMPLANTAÇÃO

### 10.1 Execução local

Com o Docker instalado, a aplicação completa é iniciada com dois comandos:

```
docker compose up -d --build
docker compose exec api php artisan db:seed
```

O primeiro sobe o banco de dados, a API e o *frontend*, e cria as tabelas; o segundo carrega os dados de demonstração. A aplicação fica disponível em `http://localhost:3000`.

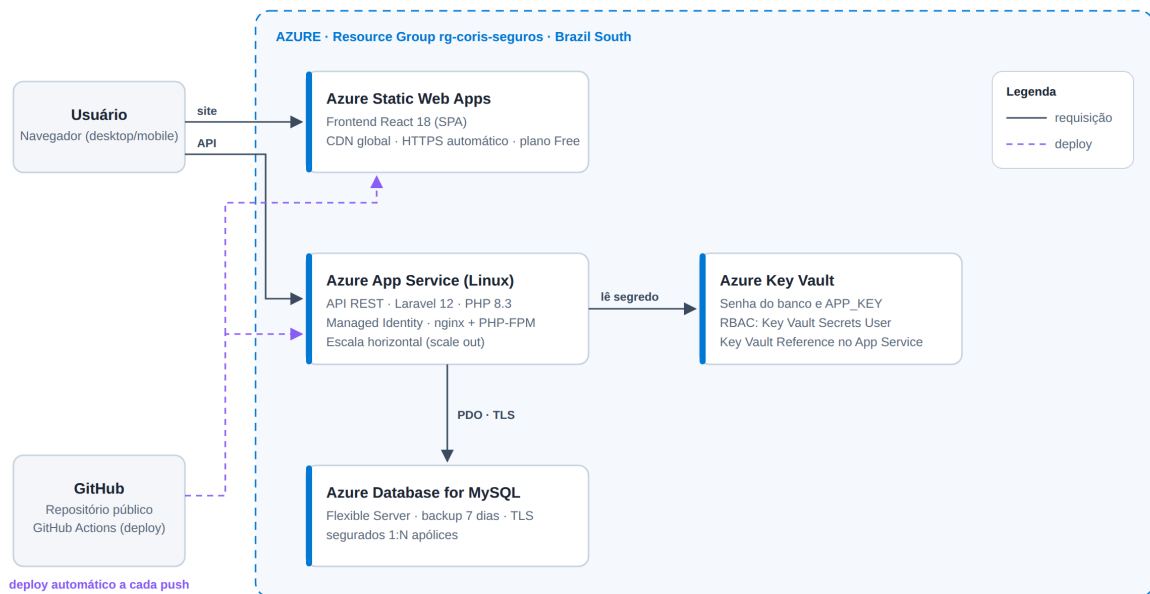
### 10.2 Implantação na Microsoft Azure

A Figura 9 apresenta o desenho da solução na Azure. O *frontend* React é publicado no **Azure Static Web Apps**, que o distribui por uma rede de entrega de conteúdo com HTTPS. A API Laravel é executada no **Azure App Service**, que permite escalar a aplicação conforme a demanda. Os dados ficam no **Azure Database for MySQL**, um banco gerenciado com cópias de segurança automáticas. As senhas e chaves ficam no **Azure Key Vault**, lidas pela API por

meio de uma identidade gerenciada, de modo que nenhum segredo fica no código. A publicação é feita pelo GitHub Actions a cada atualização do repositório.

Figura 9 – Desenho da solução na Microsoft Azure

Coris Seguros · Gestão de Apólices — arquitetura na Azure



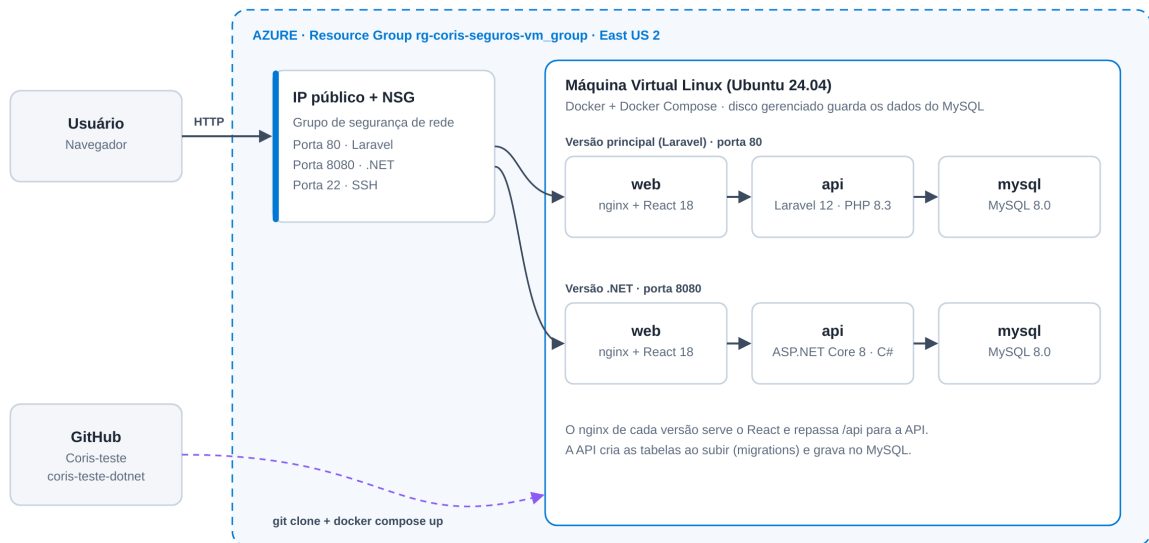
Fonte: elaborado pelo autor (2026).

Os arquivos para esse modelo estão prontos no repositório: o script de inicialização e a configuração do nginx para o App Service e um *workflow* do GitHub Actions que publica a API a cada alteração na *branch* principal. Na assinatura de avaliação usada, porém, a Azure bloqueou o App Service por falta de cota e o provisionamento do MySQL gerenciado falhou.

Por isso, a demonstração foi publicada numa **máquina virtual Linux da Azure**, que executa o mesmo Docker Compose usado no desenvolvimento (Figura 10). Na mesma máquina rodam as duas versões do projeto: a principal, em Laravel, em `http://172.172.89.245`, e a versão em .NET, em `http://172.172.89.245:8080`. O IP público e o grupo de segurança de rede liberam apenas as portas usadas. A diferença entre os dois desenhos é o modelo de serviço: na máquina virtual (IaaS) o servidor é administrado pelo desenvolvedor, enquanto no desenho proposto (PaaS) a Azure cuida do servidor, da escala e das cópias de segurança.

Figura 10 – Publicação atual na Microsoft Azure

## Coris Seguros · Gestão de Apólices — publicação atual na Azure (VM)



Fonte: elaborado pelo autor (2026).

## 11 CONSIDERAÇÕES FINAIS

O projeto atingiu o objetivo de construir uma aplicação de gestão de apólices de seguro viagem com o ciclo completo de cadastro, cálculo automático do prêmio e regras reais do negócio de seguros, como a exclusão lógica para auditoria e o armazenamento dos valores em centavos. O código foi organizado em camadas, com a aplicação dos princípios SOLID e das práticas de Clean Code identificada em pontos concretos, e o funcionamento é garantido por testes automatizados.

Como diferencial, a *dashboard* oferece a cada time as informações de que ele precisa: o marketing avalia a conversão e o retorno das campanhas, o comercial acompanha as vendas em valores e a diretoria tem uma visão consolidada do negócio.

Como trabalhos futuros, destacam-se a migração da publicação para os serviços gerenciados da Azure, a autenticação de usuários com perfis de acesso por time, o registro do histórico de alterações de cada apólice (endosso), testes automatizados no *frontend* e, com o crescimento do volume de dados, o uso de indicadores pré-calculados para acelerar a *dashboard*.

## REFERÊNCIAS

LARAVEL. **Laravel documentation**: version 12.x. Disponível em: <https://laravel.com/docs/12.x>. Acesso em: 25 set. 2026.

MARTIN, Robert C. **Código limpo**: habilidades práticas do Agile Software. Rio de Janeiro: Alta Books, 2009.

MARTIN, Robert C. **Arquitetura limpa**: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

MICROSOFT. **Documentação do Azure**. Disponível em: <https://learn.microsoft.com/pt-br/azure/>. Acesso em: 25 set. 2026.

REACT. **React documentation**. Disponível em: <https://react.dev>. Acesso em: 25 set. 2026.